

1.5-feature-analysis

1 Feature Analysis

This notebook analyses each feature individually.

```
[ ]: import polars as pl
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import numpy as np

PARQUET_PATH = "../../../data/interim/collapsed_waveform_tuning.parquet"

df = pl.read_parquet(PARQUET_PATH)

pl.Config.set_tbl_rows(-1)

[ ]: feature_cols = ['Color$', 'HeadIdx$', 'V', 'F_r', 'dt2', 'Coverage$']
value_cols = [c for c in df.columns if c.startswith("Value_")]
```

2 Colour

2.1 Basic Analysis

```
[ ]: TARGET_COL = 'Color$'
OTHER_COLS = ['HeadIdx#', 'V', 'F_r', 'dt2', 'Coverage#']

print("=" * 55)
print(f"  EDA > {TARGET_COL}")
print("=" * 55)

total      = len(df)
null_count = df[TARGET_COL].null_count()
n_unique   = df[TARGET_COL].n_unique()

print(f"\n{'Dtype:':<25} {df[TARGET_COL].dtype}")
print(f"{'Total rows:':<25} {total:,}")
print(f"{'Null count:':<25} {null_count:,} ({null_count/total*100:.2f}%)")
print(f"{'Unique values:':<25} {n_unique}")

[ ]: vc = (
    df
    .group_by(TARGET_COL)
    .agg(pl.len().alias('count'))
    .with_columns((pl.col('count') / total * 100).round(2).alias('pct (%)'))
    .sort('count', descending=True)
)

print(f"\n{'Value Counts': <45}")
print(vc)
```

2.1.1 Observations

Colour is perfectly balanced across the dataset, each having 25% of the data.

Since it's a categorical variable, it will be encoded using One-Hot encoding.

3 Coverage

```
[ ]: COL      = 'Coverage#'
GROUP_COL   = 'Color$'
WAVEFORM    = ['V', 'F_r', 'dt2']

print("=" * 55)
print(f"  EDA > {COL} (pre-set input variable)")
print("=" * 55)

total      = len(df)
null_count = df[COL].null_count()

print(f"\n{'Dtype:':<25} {df[COL].dtype}")
```

```

print(f"{'Total rows:':<25} {total:,}")
print(f"{'Null count:':<25} {null_count}")
print(f"{'Unique levels:':<25} {df[COL].n_unique()}")
print(f"{'Levels:':<25} {sorted(df[COL].drop_nulls().unique().to_list())}")
print(f"{'Min:':<25} {df[COL].min()}")
print(f"{'Max:':<25} {df[COL].max()}")

```

```

[ ]: vc = (
    df.groupby(COL)
    .agg(pl.len().alias('count'))
    .with_columns((pl.col('count') / total * 100).round(2).alias('pct (%)'))
    .sort(COL)
)
print(f"\n{'Level distribution':<45}")

with pl.Config(tbl_cols=-1):
    print(vc)

```

```

[ ]: # 3. CROSS-TAB - Coverage# × Color$
print(f"\n{'Cross-tab: Coverage# × Color$ (counts)':<55}")
crosstab = (
    df.groupby([COL, GROUP_COL])
    .agg(pl.len().alias('count'))
    .sort([COL, GROUP_COL])
    .pivot(on=GROUP_COL, index=COL, values='count')
)
print(crosstab)

```

4 Voltage, Flank Ratio, dt2

```

[ ]: colors_list = sorted(df[GROUP_COL].drop_nulls().unique().to_list())
palette = sns.color_palette("Set2", len(colors_list))

```

```

[ ]: print("=" * 60)
print(" EDA > Waveform Features: V, F_r, dt2")
print("=" * 60)

for col in WAVEFORM:
    vals = sorted(df[col].drop_nulls().unique().to_list())
    null_count = df[col].null_count()
    print(f"\n{' '*45}")
    print(f" {col}")
    print(f"{' '*45}")
    print(f" dtype      : {df[col].dtype}")
    print(f" nulls       : {null_count}")
    print(f" n_unique    : {len(vals)}")

```

```

print(f"  levels      : {vals}")
print(f"  min/max      : {df[col].min()} - {df[col].max()}")

# Value counts
vc = (
    df.groupby(col)
      .agg(pl.len().alias('count'))
      .with_columns((pl.col('count') / len(df) * 100).round(2).alias('pct_
↳(%)'))
      .sort(col)
)
print(f"\n  Level counts:")
print(vc)

```

```

[ ]: print(f"\n{'='*60}")
print("  Cross-check: do all Color$ classes share the same levels?")
print(f"{'='*60}")

for col in WAVEFORM:
    print(f"\n{col}:")
    for c in colors_list:
        vals = sorted(df.filter(pl.col(GROUP_COL) == c)[col].drop_nulls().
↳unique().to_list())
        print(f"  {c}: {vals}")

```

```

[ ]: fig = plt.figure(figsize=(18, 12))
fig.suptitle("Waveform Features - V, F_r, dt2", fontsize=14, fontweight='bold')
gs = gridspec.GridSpec(3, 3, figure=fig, hspace=0.55, wspace=0.38)

for row, col in enumerate(WAVEFORM):
    vc = (
        df.groupby(col)
          .agg(pl.len().alias('count'))
          .sort(col)
    )
    level_labels = [str(v) for v in vc[col].to_list()]
    level_counts = vc['count'].to_list()
    n_levels = len(level_labels)
    lpalette = sns.color_palette("Blues_d", n_levels)

    # 4a. Level distribution (bar chart)
    ax1 = fig.add_subplot(gs[row, 0])
    bars = ax1.bar(level_labels, level_counts, color=lpalette,
↳edgecolor='white')
    ax1.bar_label(bars, fmt='%d', padding=2, fontsize=7)
    ax1.set_title(f"{col} - Level Counts", fontweight='bold')
    ax1.set_xlabel(col)

```

```

ax1.set_ylabel("Count")
ax1.tick_params(axis='x', rotation=30)

# 4b. Count per level per Color$ - grouped bar
ax2 = fig.add_subplot(gs[row, 1])
levels_sorted = sorted(df[col].drop_nulls().unique().to_list())
x = np.arange(len(levels_sorted))
width = 0.8 / len(colors_list)
for i, c in enumerate(colors_list):
    group_counts = [
        df.filter((pl.col(col) == lvl) & (pl.col(GROUP_COL) == c)).height
        for lvl in levels_sorted
    ]
    ax2.bar(x + i * width, group_counts, width, label=c, color=palette[i],
    ↪edgecolor='white')
    ax2.set_xticks(x + width * (len(colors_list) - 1) / 2)
    ax2.set_xticklabels([str(l) for l in levels_sorted], rotation=30,
    ↪fontsize=7)
    ax2.set_title(f"{col} × Color$ balance", fontweight='bold')
    ax2.set_xlabel(col)
    ax2.set_ylabel("Count")
    ax2.legend(title=GROUP_COL, fontsize=7)

# 4c. Box plot of OTHER waveform features conditioned on this level
# Shows whether levels co-vary with each other
other_cols = [w for w in WAVEFORM if w != col]
ax3 = fig.add_subplot(gs[row, 2])
box_data = [
    df.filter(pl.col(col) == lvl)[other_cols[0]].drop_nulls().to_numpy()
    for lvl in levels_sorted
]
ax3.boxplot(box_data, labels=[str(l) for l in levels_sorted],
    ↪patch_artist=True,
            boxprops=dict(facecolor='steelblue', alpha=0.6))
ax3.set_title(f"{other_cols[0]} conditioned on {col}", fontweight='bold')
ax3.set_xlabel(col)
ax3.set_ylabel(other_cols[0])
ax3.tick_params(axis='x', rotation=30)

plt.show()

```

4.1 Observations

Voltage is mostly evenly spread in counts and across colours. F_r conditioned on V shows a slight upward trend -> might indicate that high voltage tends to pair with high F_r values.

dt2 has 5 levels, all of them perfectly balanced. Since V conditioned on dt2 is flat, they're fully independent of each other.

F_r has highly unequal counts, with 1.02 having 21570 rows, while most others have ~3600. V conditioned on F_r also shows that not all voltage levels were tested for all F_r values.

```
[ ]: # Full cross-tab of V x F_r - shows exactly which combinations exist
vf_crosstab = (
    df.groupby(['V', 'F_r'])
      .agg(pl.len().alias('count'))
      .sort(['F_r'])
      .pivot(on='V', index='F_r', values='count')
      .fill_null(0)
)
print(vf_crosstab)

# How many F_r levels have incomplete V coverage?
missing = (
    df.groupby(['V', 'F_r'])
      .agg(pl.len().alias('count'))
      .group_by('F_r')
      .agg(pl.len().alias('n_V_levels'))
      .sort('F_r')
)
print(missing)

[ ]: # Confirm the V-F_r mapping is consistent across Color$
mapping = (
    df.groupby(['V', 'F_r', 'Color$'])
      .agg(pl.len().alias('count'))
      .sort(['V', 'F_r', 'Color$'])
)
print(mapping)

# Create a composite waveform index for later use
df = df.with_columns(
    (pl.col('V').cast(pl.Utf8) + '_' + pl.col('F_r').cast(pl.Utf8)).
    <-alias('waveform_id')
)
print(f"Unique waveform operating points: {df['waveform_id'].n_unique()}")
```

4.2 Observations (cont.)

Each voltage has exactly 5 paired F_r levels, and they increase with V:

F_r	V=20	V=22	V=24	V=26	V=28	V=30
1.02						
1.08						
1.09						
1.095						

F_r	V=20	V=22	V=24	V=26	V=28	V=30
1.1						
1.105						
1.11						
1.14						
1.16						
1.17						
1.18						
1.19						
1.2						
1.23						
1.245						
1.26						
1.275						
1.29						
1.3						
1.32						
1.34						
1.36						
1.379						

5 HeadId

```
[ ]: COL      = 'HeadIdx#'
GROUP_COL = 'Color$'
INPUTS    = ['waveform_idx', 'dt2', 'Coverage#']

# 1. BASICS
print("=" * 55)
print(f"  EDA > {COL}")
print("=" * 55)

total      = len(df)
null_count = df[COL].null_count()
n_unique   = df[COL].n_unique()
vals       = sorted(df[COL].drop_nulls().unique().to_list())

print(f"\n{'Dtype:':<25} {df[COL].dtype}")
print(f"{'Nulls:':<25} {null_count}")
print(f"{'n_unique:':<25} {n_unique}")
print(f"{'Levels:':<25} {vals}")
print(f"{'Min/Max:':<25} {df[COL].min()} / {df[COL].max()}")

[ ]: vc = (
    df.groupby(COL)
    .agg(pl.len().alias('count'))
```

```

        .with_columns((pl.col('count') / total * 100).round(2).alias('pct (%)'))
        .sort(COL)
    )
    print(f"\n{'Value Counts': <45}")
    print(vc)

```

```

[ ]: # Is HeadIdx# balanced across Color$?
print(f"\n{'Cross-tab: HeadIdx# × Color$': <55}")
ct_color = (
    df.group_by([COL, GROUP_COL])
      .agg(pl.len().alias('count'))
      .sort([COL, GROUP_COL])
      .pivot(on=GROUP_COL, index=COL, values='count')
      .fill_null(0)
)
print(ct_color)

# Is HeadIdx# balanced across waveform operating points?
print(f"\n{'Cross-tab: HeadIdx# × waveform_idx (sample)': <55}")
ct_wf = (
    df.group_by([COL, 'waveform_idx'])
      .agg(pl.len().alias('count'))
      .sort([COL, 'waveform_idx'])
      .pivot(on='waveform_idx', index=COL, values='count')
      .fill_null(0)
)
print(ct_wf)

```

```

[ ]: # 4. KEY QUESTION - does HeadIdx# co-vary with any input?
print(f"\n{'HeadIdx# × dt2': <45}")
ct_dt2 = (
    df.group_by([COL, 'dt2'])
      .agg(pl.len().alias('count'))
      .sort([COL, 'dt2'])
      .pivot(on='dt2', index=COL, values='count')
      .fill_null(0)
)
print(ct_dt2)

print(f"\n{'HeadIdx# × Coverage# (n_unique Coverage per Head)': <55}")
cov_per_head = (
    df.group_by(COL)
      .agg(pl.col('Coverage#').n_unique().alias('n_coverage_levels'))
      .sort(COL)
)
print(cov_per_head)

```

```

[ ]: # 5. PLOTS
n_heads = df[COL].n_unique()
heads = sorted(df[COL].drop_nulls().unique().to_list())
colors = sorted(df[GROUP_COL].drop_nulls().unique().to_list())
palette = sns.color_palette("Set2", len(colors))
hpalette = sns.color_palette("Blues_d", n_heads)

fig = plt.figure(figsize=(40, 20))
fig.suptitle(f"EDA - {COL}", fontsize=14, fontweight='bold')
gs = gridspec.GridSpec(2, 3, figure=fig, hspace=0.2, wspace=0.2)

# 5a. Level counts
ax1 = fig.add_subplot(gs[0, 0])
bars = ax1.bar([str(h) for h in heads], vc['count'].to_list(),
               color=hpalette, edgecolor='white')
ax1.bar_label(bars, fmt='%d', padding=3, fontsize=8)
ax1.set_title("Samples per HeadIdx#", fontweight='bold')
ax1.set_xlabel(COL)
ax1.set_ylabel("Count")

# 5b. HeadIdx# × Color$ grouped bar
ax2 = fig.add_subplot(gs[0, 1])
x = np.arange(n_heads)
width = 0.8 / len(colors)
for i, c in enumerate(colors):
    counts = [
        df.filter((pl.col(COL) == h) & (pl.col(GROUP_COL) == c)).height
        for h in heads
    ]
    ax2.bar(x + i * width, counts, width, label=c, color=palette[i],
           edgecolor='white')
ax2.set_xticks(x + width * (len(colors) - 1) / 2)
ax2.set_xticklabels([str(h) for h in heads], rotation=30, fontsize=8)
ax2.set_title("HeadIdx# × Color$ balance", fontweight='bold')
ax2.set_xlabel(COL)
ax2.set_ylabel("Count")
ax2.legend(title=GROUP_COL, fontsize=8)

# 5c. HeadIdx# × waveform_idx heatmap
ax3 = fig.add_subplot(gs[0, 2])
heat = np.array([
    df.filter((pl.col(COL) == h) & (pl.col('waveform_idx') == w)).height
    for w in range(df['waveform_idx'].n_unique())
    for h in heads
])
sns.heatmap(heat, ax=ax3, cmap='YlOrRd',
            xticklabels=range(df['waveform_idx'].n_unique()),

```

```

        yticklabels=[str(h) for h in heads],
        linewidths=0.3)
ax3.set_title("HeadIdx# × waveform_idx", fontweight='bold')
ax3.set_xlabel("waveform_idx")
ax3.set_ylabel(COL)
ax3.tick_params(axis='x', labelsiz=6, rotation=90)

# 5d. HeadIdx# × dt2 heatmap
ax4 = fig.add_subplot(gs[1, 0])
dt2_levels = sorted(df['dt2'].drop_nulls().unique().to_list())
heat_dt2 = np.array([
    [df.filter((pl.col(COL) == h) & (pl.col('dt2') == d)).height
     for d in dt2_levels]
    for h in heads
])
sns.heatmap(heat_dt2, annot=True, fmt='d', ax=ax4, cmap='YlOrRd',
            xticklabels=[str(d) for d in dt2_levels],
            yticklabels=[str(h) for h in heads],
            linewidths=0.5)
ax4.set_title("HeadIdx# × dt2", fontweight='bold')
ax4.set_xlabel("dt2")
ax4.set_ylabel(COL)

# 5e. HeadIdx# × Coverage# heatmap
ax5 = fig.add_subplot(gs[1, 1])
cov_levels = sorted(df['Coverage#'].drop_nulls().unique().to_list())
heat_cov = np.array([
    [df.filter((pl.col(COL) == h) & (pl.col('Coverage#') == c)).height
     for c in cov_levels]
    for h in heads
])
sns.heatmap(heat_cov, ax=ax5, cmap='YlOrRd',
            xticklabels=[str(c) for c in cov_levels],
            yticklabels=[str(h) for h in heads],
            linewidths=0.3)
ax5.set_title("HeadIdx# × Coverage#", fontweight='bold')
ax5.set_xlabel("Coverage#")
ax5.set_ylabel(COL)
ax5.tick_params(axis='x', labelsiz=7, rotation=90)

# 5f. Imbalance ratio
ax6 = fig.add_subplot(gs[1, 2])
max_count = vc['count'].max()
ratios = [c / max_count for c in vc['count'].to_list()]
ax6.barh([str(h) for h in heads], ratios, color=hpalette, edgecolor='white')
ax6.axvline(0.1, color='red', linestyle='--', linewidth=1, label='10%_
↳threshold')

```

```
ax6.set_title("Imbalance Ratio", fontweight='bold')
ax6.set_xlabel("Ratio (1.0 = majority)")
ax6.legend(fontsize=8)

plt.show()
```

5.1 Observations

Most chips have 3600 sample rows, except for chips 5 through 12, which indicates not all possible waveform combinations were tested on all chips.

Chip 19 has 3540 sample rows, also indicating the lack of possibly one waveform configuration.

Chips 5 through 12 are missing waveform_idx 7, 9, 11, 13. They also have fewer dt2 combinations tested on them. On those same chips, the coverage level is significantly lower.

```
[ ]: incomplete_heads = df.groupby('HeadIdx#').agg(
    pl.col('waveform_idx').n_unique().alias('n_waveforms')
).filter(pl.col('n_waveforms') < df['waveform_idx'].n_unique()).sort('HeadIdx#')
print(incomplete_heads)
```

6 Summary

6.0.1 Dataset Structure

A fully designed experiment testing inkjet printhead chips across multiple waveform and print conditions. Every row represents one (chip, waveform, dt2, coverage, color) configuration.

6.0.2 Features

Feature	Type	Levels	Role	Notes
Color\$	Categorical	4 (C,M,Y,K)	Label	Perfectly balanced, ~25% each
HeadIdx#	Discrete	30 chips	Core feature	Chip-to-chip variation is the signal
V + F_r	Discrete	30 operating points	Composite input	Not independent
dt2	Discrete	5 levels	Input	Fully independent, perfectly balanced
Coverage#	Discrete	30 levels	Input	Pre-set, fully independent

6.0.3 Key Findings

V and F_r are a single composite feature. Each voltage has exactly 4 paired F_r values representing a resonance sweep. F_r = 1.02 is a baseline tested at all voltages.

Every (chip, waveform, dt2, coverage, color) combination was tested, with two exceptions:
- Chips 5–12 are missing waveform configurations (V=22/24) - Chip 19 is missing one waveform configuration

No nulls, no invalid values, no outliers

6.0.4 Modelling Implications

- Stratify train/test split by HeadIdx# to ensure all chips are represented in both sets
- Chips 5–12 have no observations at certain waveform configurations, so model predictions for those chip/waveform combinations will be interpolation from other chips
- All inputs are discrete, so tree-based models (Random Forest, XGBoost) are a natural fit; if using a neural network, treat all inputs as embeddings rather than raw numerics